Types of Errors:

- **SyntaxError**: Errors due to malformed code (e.g., typos, mismatched brackets). Detected before execution.
- **ReferenceError**: Occurs when accessing undeclared variables or functions at runtime.
- **TypeError**: Happens when a value doesn't match the expected type (e.g., assigning to a constant).
- **RangeError**: Triggered when a value falls outside an allowable range (e.g., invalid array length).
- **Other Errors**: EvalError, InternalError, URIError (rarely encountered).

2. **Error Handling Constructs**:
   - **try–catch Statement**: Allows catching and handling runtime errors to prevent program termination.
   - **Conditional Exception Handling**: Uses the instanceof operator to handle specific error types.
   - **finally Statement**: Executes regardless of whether an error occurred or not.
3. **Custom Errors**:
   - Using the throw statement to create and handle custom exceptions.

Detailed Overview:

Basic Error Types:

1. **SyntaxError**:
   - Caused by code that cannot be parsed.
   - Example:

javascript

Copy code

```
"use strict";

iff (true) { console.log("true"); } // SyntaxError: Unexpected token '{'
```

2. **ReferenceError**:
   - Triggered when referencing undeclared variables or functions.
   - Example:

javascript

Copy code

```
let a = b; // ReferenceError: b is not defined
fun();     // ReferenceError: fun is not defined
```

3. **TypeError**:
    - Occurs when a value is used in an incompatible manner.
    - Example:

javascript

Copy code

```
const someConst = 5;
someConst = 7; // TypeError: Assignment to constant variable.
```

4. **RangeError**:
    - Happens when a value falls outside the acceptable range.
    - Example:

javascript

Copy code

```
let arr = Array(-1); // RangeError: Invalid array length
```

---

try–catch Statement:

- Syntax:

javascript

Copy code

```
try {
  // Code to attempt
} catch (error) {
  // Handle error
}
```

- Example:

javascript

Copy code

```
try {
  let a = b;
} catch (error) {
  console.log("Caught: " + error);
}
console.log("Program continues...");
```

Key Notes:

- SyntaxErrors cannot be caught using try–catch since the program doesn't execute.
- Variables declared with let inside try are not accessible in catch.

Conditional Exception Handling:

- Use instanceof to check error types.
- Example:

javascript

Copy code

```
try {
  let a = b;
} catch (error) {
  if (error instanceof ReferenceError) {
    console.log("Reference error occurred.");
  } else {
    console.log("Other error: " + error);
  }
}
```

---

finally Statement:

- Executes after try and catch regardless of the outcome.
- Example:

javascript

Copy code

```javascript
try {
   let a = b; // ReferenceError
} catch (error) {
   console.log("Caught: " + error);
} finally {
   console.log("Execution completed.");
}
```

Why Use finally?

- For cleanup tasks or ensuring specific actions always occur.